

LAB RESOURCE UTILIZATION PLATFORM

Project Documentation

Milestone 1

Includes: Project Understanding & Objectives · Problem Statement · User Roles & Permissions · Functional & Non-Functional Requirements · Workflow Diagram · System Architecture Diagram · Database Schema (ER Diagram) · UI Wireframes

August 2026

Table of Contents

Table of Contents	2
1. Project Understanding and Objectives	3
1.1 Overview	3
1.2 Background	3
1.3 Objectives	3
1.4 Scope (Week 1)	3
2. Problem Statement.....	3
2.1 Proposed Solution.....	4
3. User Roles and Permissions.....	4
3.1 Permission Matrix	5
4. Functional and Non-Functional Requirements.....	5
4.1 Functional Requirements	5
4.2 Non-Functional Requirements.....	6
5. Workflow Diagram.....	6
6. System Architecture Diagram.....	7
6.1 Technology Stack	8
7. Database Schema (ER Diagram)	8
7.1 Key Entities	9
7.2 Notable Constraints	10
8. UI Wireframes	10
8.1 Screen Notes.....	10
9. Summary	11

1. Project Understanding and Objectives

1.1 Overview

The Lab Resource Utilization Platform is a web-based system designed to help academic institutions manage, book, and track the usage of laboratory equipment and resources. The platform connects students, faculty, lab technicians, and administrators through a single portal that streamlines equipment discovery, booking, approval, maintenance tracking, and usage reporting.

The system follows a three-tier architecture: a React-based single page application (SPA) for the frontend, a REST API backend (Spring Boot / Java, based on the current project scaffolding) for business logic, and a PostgreSQL relational database for persistent storage.

1.2 Background

Educational institutions typically operate multiple laboratories across departments, each equipped with shared instruments and machinery (e.g., microscopes, spectrometers, 3D printers, computing clusters). Today, usage of this equipment is often coordinated manually — through paper registers, spreadsheets, or informal verbal requests — which leads to double-booking, underutilized assets, undocumented maintenance history, and poor visibility into which resources are most (or least) used.

1.3 Objectives

- Provide a centralized catalog of labs, departments, and equipment across one or more institutions.
- Allow students and faculty to search, view availability, and request bookings for lab equipment.
- Allow administrators and technicians to approve/reject booking requests and manage scheduling conflicts.
- Track equipment maintenance history and automatically flag equipment under maintenance as unavailable.
- Provide real-time notifications for booking status changes, approvals, and maintenance alerts.
- Generate utilization reports to help institutions make data-driven decisions about resource allocation and future procurement.
- Support role-based access control so each user type only sees the functionality relevant to them.
- Deliver a responsive, intuitive UI accessible from desktop and mobile browsers.

1.4 Scope (Week 1)

Week 1 focuses on requirements gathering and design: understanding the problem, defining roles and permissions, capturing functional and non-functional requirements, and producing the foundational diagrams (workflow, architecture, ER diagram) and UI wireframes that will guide implementation in subsequent weeks.

2. Problem Statement

Academic and research institutions manage a wide variety of expensive, shared lab equipment across multiple departments. In the absence of a centralized digital system, institutions face the following recurring problems:

- No single source of truth for what equipment exists, where it is located, and whether it is currently available.

- Manual, paper-based or spreadsheet-based booking processes that are slow, error-prone, and hard to audit.
- Double-bookings and scheduling conflicts due to lack of real-time availability visibility.
- No structured approval workflow — requests are approved informally with no traceable record.
- Maintenance schedules are tracked inconsistently, sometimes resulting in equipment being booked while under repair.
- Department and institution administrators lack visibility into utilization trends, making it difficult to justify new purchases or identify underused assets.
- Users are not notified proactively about approvals, rejections, or upcoming maintenance windows.

2.1 Proposed Solution

The Lab Resource Utilization Platform addresses these problems by providing a single web application where all stakeholders — students, faculty, lab technicians, and administrators — can view equipment availability, submit and manage booking requests, track maintenance, and generate utilization reports, all governed by a role-based permission model and backed by a relational database that enforces data integrity.

3. User Roles and Permissions

The schema defines a generic roles table, allowing role definitions to be configured rather than hard-coded. Based on the platform's intended workflow, four primary roles are defined below.

Role	Description	Key Permissions
Student / Faculty (End User)	Registers, browses equipment, and books lab resources for coursework or research.	Register/Login; view labs & equipment catalog; create booking requests; view own booking history & status; receive notifications; update own profile.
Lab Technician	Manages day-to-day equipment condition, maintenance scheduling, and department-level bookings.	All Student/Faculty permissions; approve/reject bookings for assigned department; log and schedule maintenance; update equipment status (Available/In Use/Under Maintenance).
Department / Institution Admin	Oversees all labs and equipment within a department or institution.	All Technician permissions; manage equipment inventory (add/edit/retire); manage users within institution; view utilization reports; manage department records.
Super Admin	Platform-wide administrator with cross-institution visibility.	All Admin permissions; manage institutions and departments; manage roles; manage all users across institutions; access system-wide analytics and reports.

3.1 Permission Matrix

Feature / Action	Student/Faculty	Technician	Admin	Super Admin
------------------	-----------------	------------	-------	-------------

Feature / Action	Student/Faculty	Technician	Admin	Super Admin
Browse equipment catalog	Yes	Yes	Yes	Yes
Create booking request	Yes	Yes	Yes	Yes
Approve/reject bookings	No	Yes (own dept.)	Yes	Yes
Add/edit/retire equipment	No	Limited	Yes	Yes
Schedule maintenance	No	Yes	Yes	Yes
View own booking history	Yes	Yes	Yes	Yes
View utilization reports	No	Limited	Yes	Yes
Manage users	No	No	Yes (own inst.)	Yes
Manage institutions/departments	No	No	No	Yes

4. Functional and Non-Functional Requirements

4.1 Functional Requirements

ID	Requirement	Priority
FR-01	Users shall be able to register and log in with email and password.	High
FR-02	The system shall support role-based dashboards (Student/Faculty, Technician, Admin, Super Admin).	High
FR-03	Users shall be able to browse labs and equipment, filterable by category, department, and availability status.	High
FR-04	Users shall be able to submit a booking request specifying equipment, date, and time slot.	High
FR-05	The system shall prevent overlapping bookings for the same equipment and time slot.	High
FR-06	Technicians/Admins shall be able to approve or reject booking requests with optional remarks.	High
FR-07	The system shall send notifications to users on booking approval, rejection, or upcoming maintenance.	Medium
FR-08	Technicians shall be able to log and schedule equipment maintenance, updating equipment status accordingly.	High
FR-09	Admins shall be able to add, edit, and retire equipment records, including category, cost, and warranty details.	Medium
FR-10	The system shall generate utilization reports (bookings per equipment, usage hours, downtime) filterable by date range.	Medium
FR-11	Users shall be able to view and update their own profile information.	Low

ID	Requirement	Priority
FR-12	The system shall maintain an audit trail of booking status changes (created, approved, rejected).	Medium
FR-13	Super Admins shall be able to manage institutions and departments.	Medium

4.2 Non-Functional Requirements

Category	Requirement
Performance	Page loads and API responses for common actions (catalog browse, booking submission) shall complete within 2 seconds under normal load.
Scalability	The system shall support multiple institutions, departments, and thousands of equipment/booking records without redesign of the schema.
Security	Passwords shall be stored using a secure hashing algorithm; sessions shall be authenticated via tokens (e.g., JWT); role-based access control shall be enforced on every API endpoint.
Usability	The UI shall be responsive (desktop and mobile) and follow a consistent, accessible design system (Bootstrap 5).
Availability	The system shall target 99% uptime during institutional working hours.
Maintainability	The codebase shall follow a layered architecture (Controller-Service-Repository) to keep business logic testable and decoupled from the API and data layers.
Data Integrity	The database shall enforce referential integrity via foreign key constraints across users, equipment, bookings, and related entities.
Auditability	All booking and maintenance status changes shall be timestamped and traceable to a user.
Compatibility	The frontend shall support the latest versions of major browsers (Chrome, Firefox, Edge, Safari).

5. Workflow Diagram

The diagram below illustrates the end-to-end workflow: a user registers/logs in, is routed based on their role, and either browses and books equipment (Student/Faculty path) or reviews and manages bookings and maintenance (Technician/Admin path).

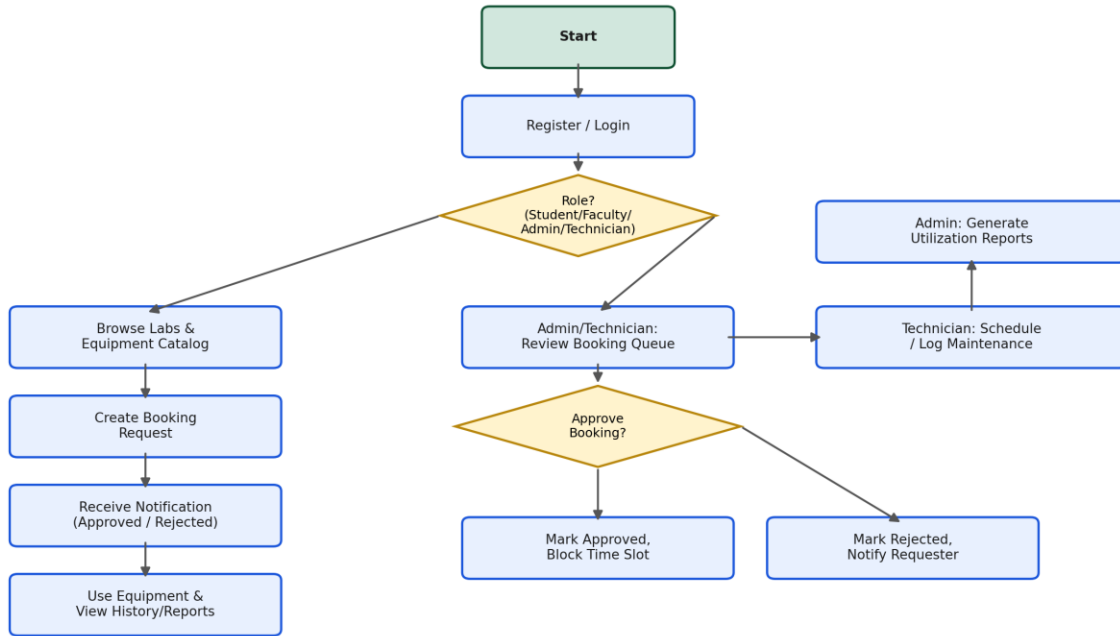
User Workflow — Lab Resource Utilization Platform

Figure 1: User workflow across roles — registration, booking, approval, and reporting.

6. System Architecture Diagram

The platform follows a layered, three-tier architecture. The React SPA (client layer) communicates with a Spring Boot REST API (application layer) over HTTPS using JWT-authenticated requests. The application layer delegates persistence to a PostgreSQL database (data layer), with cross-cutting concerns such as security, notifications, and scheduled maintenance jobs applied across all layers.

System Architecture — Lab Resource Utilization Platform

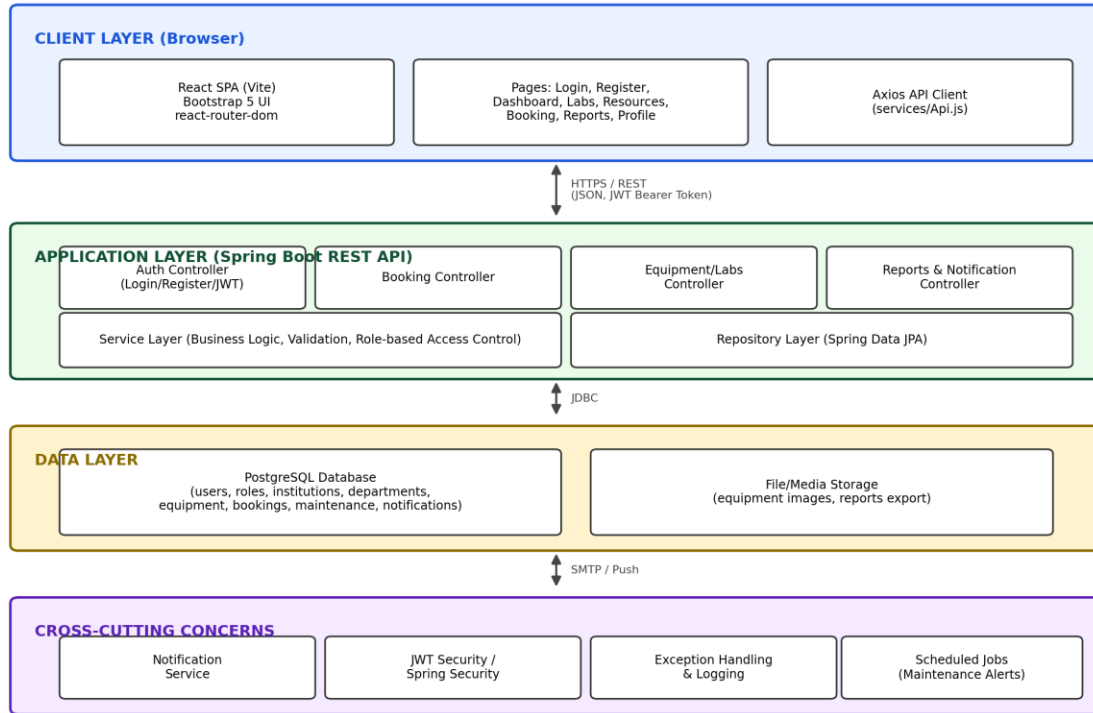


Figure 2: Three-tier system architecture — client, application, and data layers.

6.1 Technology Stack

Layer	Technology
Frontend	React 18 (Vite), React Router DOM, Bootstrap 5, Axios, React Icons
Backend	Java / Spring Boot REST API (Controller-Service-Repository layers), Spring Security (JWT)
Database	PostgreSQL (relational schema with referential integrity constraints)
Build Tooling	Vite (frontend), Maven (backend, per .gitignore/target structure)
Version Control	Git, with configured .gitattributes and .gitignore

7. Database Schema (ER Diagram)

The database schema (PostgreSQL) consists of nine core tables: roles, institutions, departments, users, equipment_categories, equipment, bookings, maintenance, and notifications. Foreign key constraints enforce referential integrity across institutions, departments, users, and equipment.

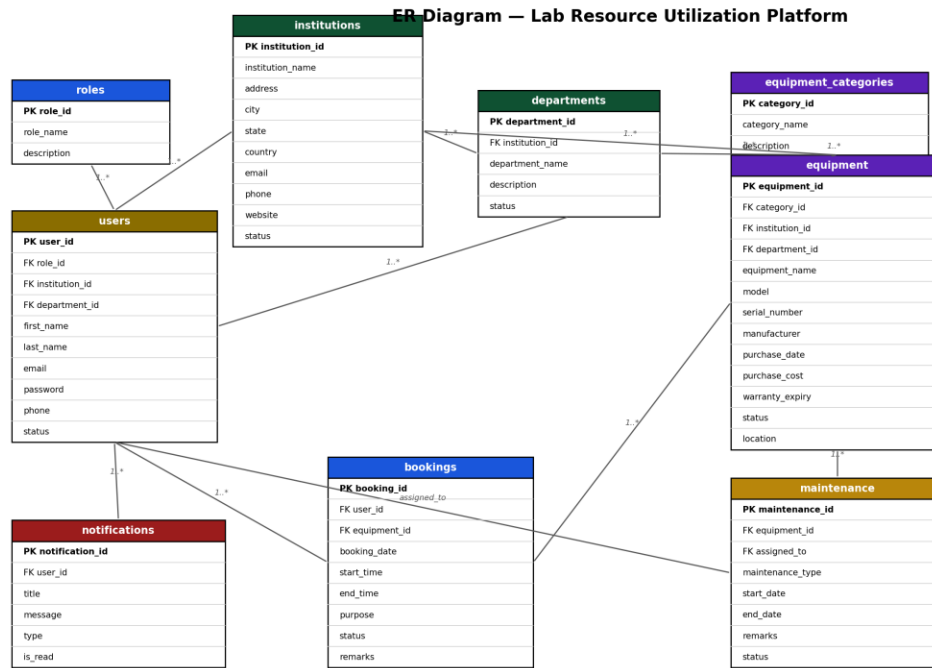


Figure 3: Entity-Relationship diagram derived from schema.sql.

7.1 Key Entities

Table	Purpose
roles	Defines available user roles (Student/Faculty, Technician, Admin, Super Admin).
institutions	Top-level organization owning departments, users, and equipment.
departments	Sub-division of an institution; owns equipment and users.
users	All platform users, linked to a role, institution, and department.
equipment_categories	Classification of equipment (e.g., Microscopes, 3D Printers, Computing).
equipment	Individual lab assets with status, cost, warranty, and location details.
bookings	Booking requests linking a user to equipment for a specific date/time slot, with status.
maintenance	Maintenance records for equipment, optionally assigned to a technician (user).
notifications	System-generated notifications delivered to users (booking/maintenance events).

7.2 Notable Constraints

- users.email is unique; equipment.serial_number is unique.
- Cascading delete from institutions to departments (ON DELETE CASCADE).
- All booking and maintenance records reference equipment and users via foreign keys.
- Indexes are defined on users.email, users.role_id, users.institution_id, equipment.department_id, bookings.user_id, and bookings.equipment_id to optimize common lookup and join queries.

8. UI Wireframes

Low-fidelity wireframes for the six primary screens implemented in the React frontend (routes defined in App.jsx): Login, Dashboard, Labs/Equipment Catalog, Booking, Reports, and Profile. Each screen shares a common Navbar/Sidebar and Footer, styled with Bootstrap 5.

UI Wireframes — Lab Resource Utilization Platform



Figure 4: Wireframes for Login, Dashboard, Equipment Catalog, Booking, Reports, and Profile screens.

8.1 Screen Notes

Screen	Route	Key Elements
Login	/	Email/password form, link to Register.
Register	/register	New user sign-up form (not pictured; mirrors Login layout with additional fields).
Dashboard	/dashboard	Summary cards (equipment count, active bookings, pending approvals, maintenance), recent bookings table, usage chart.

Screen	Route	Key Elements
Labs / Resources	/labs, /resources	Searchable/filterable equipment grid with status badges and a Book Now action.
Booking	/booking	Form to select lab, equipment, date/time, and purpose; submits a booking request.
Reports	/reports	Filters (date range, lab, equipment), utilization and booking trend charts, detailed report table, export option.
Profile	/profile	User details form and password change action.

9. Summary

This document captures the Week 1 deliverables for the Lab Resource Utilization Platform: a clear statement of the problem being solved, defined user roles and permissions, functional and non-functional requirements, and the foundational design artifacts — workflow diagram, system architecture diagram, database ER diagram, and UI wireframes — that will guide backend and frontend implementation in the following weeks.